

EXPERIMENT-13

A)Aim: Write MongoDB queries to create and drop databases and collection through Command Prompt
Description:

1. Databases:A Database is a physical container for collections. Each database has its own set of files on the file system. A single MongoDB server typically holds multiple databases.

- **Create:** Use the use <name> command. It stays "virtual" until you save data.
- **Drop:** Use db.dropDatabase() to remove the current database and its associated data files.

2. Collections:A Collection is a group of MongoDB documents. It is the equivalent of a "table" in a relational database (like MySQL), but it does not enforce a strict schema.

- **Create:** Use db.createCollection("name") or simply insert a document to create it automatically.
- **Drop:** Use db.collectionName.drop() to remove the collection and all documents within it.

Program:**VIEW ALL DATABASES**

```
test> show dbs
```

Output:

```
loginDB> show dbs
Student  68.00 KiB
Student2 76.00 KiB
admin    40.00 KiB
config   108.00 KiB
local    80.00 KiB
loginDB  64.00 KiB
```

CREATE A NEW OR SWITCH DATABASE

```
test> use student
```

Output:

```
switched to db student
```

VIEW CURRENT DATABASE

```
student1> db
```

Output:

```
student
```

DROP DATABASE

```
student> db.dropDatabase()
```

Output:

```
{ ok: 1, dropped: 'student' }
```

CREATE DATABASE

```
student1> use student
```

Output:

```
switched to db student
```

CREATE COLLECTIONS

```
student> db.createCollection('stu');
```

Output:

```
{ ok: 1 }
student> db.createCollection('stu1');
{ ok: 1 }
```

SHOW COLLECTIONS

```
student2> show collections;
```

Output:

```
stu
stu1
```

B) Aim: Write MongoDB queries to work with records using find(), limit, sort, CreateIndex(), aggregate().

Description:**1. Retrieval & Ordering**

- **find()**: Retrieves documents. Use find({}) for all or find({key: value}) to filter.
- **limit(n)**: Caps the result set at **n** documents. Useful for pagination.
- **sort({field: 1})**: Orders results. Use **1** for ascending and **-1** for descending.

2. Performance

- **createIndex()**: Creates a "lookup list" for a field. It prevents slow full-collection scans and significantly boosts query speed.

3. Data Analysis

- **aggregate()**: Processes data through a **pipeline** (sequence of stages). It is used for complex tasks like calculating totals, averages, or grouping data.

Programs:**find()**

```
student> db.stu.find().pretty();
```

Output:

```
{
  _id: ObjectId('695e2316c9888e51762a93c5'),
  sno: 1,
  sname: 'Kiran',
  dept: 'cse'
},
{
  _id: ObjectId('695e2316c9888e51762a93c6'),
  sno: 2,
  sname: 'Vamsi',
  dept: 'it'
},
{
  _id: ObjectId('695e2316c9888e51762a93c7'),
  sno: 3,
```

```

    sname: 'Prshant',
    dept: 'civil'
  },
  {
    _id: ObjectId('695e2316c9888e51762a93c8'),
    sno: 4,
    sname: 'Harsha',
    dept: 'mech'
  },
  {
    _id: ObjectId('695e2316c9888e51762a93c9'),
    sno: 5,
    sname: 'Deepak',
    dept: 'cse'
  },
]

```

LIMIT THE NO OF ROWS IN OUTPUT

Output:

```

student> db.stu.find().limit(2);
[
  {
    _id: ObjectId('695e2316c9888e51762a93c5'),
    sno: 1,
    sname: 'Kiran',
    dept: 'cse'
  },
  {
    _id: ObjectId('695e2316c9888e51762a93c6'),
    sno: 2,
    sname: 'Vams',
    dept: 'it'
  }
]

```

COUNT THE NO OF ROWS IN THE OUTPUT

```
student> db.stu.find().count();
```

Output:

```
10
```

Sort()

```
student> db.stu.find().sort({"sname":1})
```

Output:

```

},
{

```

```

    _id: ObjectId('695e2316c9888e51762a93c9'),
    sno: 5,
    sname: 'Deepak',
    dept: 'cse'
  },
  {
    _id: ObjectId('695e2316c9888e51762a93c8'),
    sno: 4,
    sname: 'Harsha',
    dept: 'mech'
  },
  {
    _id: ObjectId('695e2316c9888e51762a93c5'),
    sno: 1,
    sname: 'Kiran',
    dept: 'cse'
  },
  {
    _id: ObjectId('695e2316c9888e51762a93c7'),
    sno: 3,
    sname: 'Prashant',
    dept: 'civil'
  },
  {
    _id: ObjectId('695e2316c9888e51762a93c6'),
    sno: 2,
    sname: 'Vamsi',
    dept: 'it' }

```

createIndex()

```
student> db.stu.createIndex({sname:"text"})
```

Output:

```
sname_text
```

Aggregate()

```
student> db.stu.aggregate({$match:{sname:"rani"}}).pretty()
```

Output:

```

[
  {
    _id: ObjectId('695e2316c9888e51762a93c5'),
    sno: 1,
    sname: 'Kiran',
    dept: 'cse'
  }
]

```

```
]
```

Sum

```
student> db.stu.aggregate([{ $group:{_id:null,Total_Marks:{$sum:"$m1"}}}]).pretty()  
[ { _id: null, Total_Marks: 312 } ]
```

Avg

```
student> db.stu.aggregate([{ $group:{_id:null,Average_Marks:{$avg:"$m1"}}}]).pretty()  
[ { _id: null, Average_Marks: 31.2 } ]
```

Min

```
student> db.stu.aggregate([{ $group:{_id:null,Minimum_Mark:{$min:"$m1"}}}]).pretty()  
[ { _id: null, Minimum_Mark: 14 } ]
```

Max

```
student> db.stu.aggregate([{ $group:{_id:null,Maximum_Mark:{$max:"$m1"}}}]).pretty()  
[ { _id: null, Maximum_Mark: 50 } ]
```

Count

```
student> db.stu.aggregate([{ $count:"No_of_students"}]) .pretty()  
[ { No_of_students: 10 } ]
```